

底盘IMU通信协议规范

版本: 1.0.0

日期: 2026-01-31

适用于: UniRobot 底盘MCU开发

1. 概述

本文档定义了底盘MCU向上位机发送IMU数据的通信协议。

1.1 系统架构



2. 消息定义

2.1 CHASSIS_IMU (0x15) - 底盘IMU数据

方向: MCU → 上位机

频率: 建议 50-100 Hz

Payload 大小: 36 Bytes

```
#pragma pack(push, 1)
struct ChassisImuData {
    float roll;    // [Offset 0]  横滚角 (rad)
    float pitch;   // [Offset 4]  俯仰角 (rad)
    float yaw;     // [Offset 8]  偏航角 (rad)
    float gx;      // [Offset 12] X轴角速度 (rad/s)
    float gy;      // [Offset 16] Y轴角速度 (rad/s)
    float gz;      // [Offset 20] Z轴角速度 (rad/s)
    float ax;      // [Offset 24] X轴加速度 (m/s²)
    float ay;      // [Offset 28] Y轴加速度 (m/s²)
    float az;      // [Offset 32] Z轴加速度 (m/s²)
}; // Total: 36 bytes
#pragma pack(pop)
```

2.2 数据单位

字段	单位	说明
roll, pitch, yaw	rad	欧拉角，范围 $-\pi \sim \pi$
gx, gy, gz	rad/s	角速度
ax, ay, az	m/s ²	线加速度

2.3 坐标系定义

采用 右手坐标系 (ROS REP-103):

- X: 前向
- Y: 左向
- Z: 上向

3. 帧格式

与现有底盘通信协议一致：

HEAD	TYPE	LENGTH	PAYLOAD	CRC16	TAIL
0x5A	0x15	0x24	36 Bytes	2B	0xA5

完整帧长度: $3 + 36 + 3 = 42$ Bytes

4. MCU实现示例

4.1 发送IMU数据

```

#include <stdint.h>
#include <string.h>

#define FRAME_HEAD 0x5A
#define FRAME_TAIL 0xA5
#define MSG_TYPE_CHASSIS_IMU 0x15

// 发送IMU数据帧
void send_imu_frame(UART_HandleTypeDef *huart,
                   float roll, float pitch, float yaw,
                   float gx, float gy, float gz,
                   float ax, float ay, float az) {
    uint8_t buffer[42];
    uint8_t idx = 0;

    // 帧头
    buffer[idx++] = FRAME_HEAD;
    buffer[idx++] = MSG_TYPE_CHASSIS_IMU;
    buffer[idx++] = 36; // Payload长度

    // Payload (9个float)
    memcpy(&buffer[idx], &roll, 4); idx += 4;
    memcpy(&buffer[idx], &pitch, 4); idx += 4;
    memcpy(&buffer[idx], &yaw, 4); idx += 4;
    memcpy(&buffer[idx], &gx, 4); idx += 4;
    memcpy(&buffer[idx], &gy, 4); idx += 4;
    memcpy(&buffer[idx], &gz, 4); idx += 4;
    memcpy(&buffer[idx], &ax, 4); idx += 4;
    memcpy(&buffer[idx], &ay, 4); idx += 4;
    memcpy(&buffer[idx], &az, 4); idx += 4;

    // CRC16 (计算范围: HEAD + TYPE + LENGTH + PAYLOAD)
    uint16_t crc = calculate_crc(buffer, idx);
    buffer[idx++] = crc & 0xFF; // 低字节
    buffer[idx++] = (crc >> 8) & 0xFF; // 高字节

    // 帧尾
    buffer[idx++] = FRAME_TAIL;

    // 发送
    HAL_UART_Transmit(huart, buffer, idx, HAL_MAX_DELAY);
}

```

4.2 CRC计算

使用与底盘相同的CRC-16算法（参考 `algorithmOfCRC.c`）。

5. ROS2 话题

上位机解析后发布到：

话题	类型	说明
/chassis_imu	sensor_msgs/Imu	IMU数据

5.1 消息字段映射

```
sensor_msgs/Imu:
  header:
    stamp: 上位机接收时间
    frame_id: "chassis_imu_link"
  orientation: 从(roll,pitch,yaw)计算的四元数
  angular_velocity: (gx, gy, gz)
  linear_acceleration: (ax, ay, az)
```

6. 测试验证

6.1 使用ROS2命令行

```
# 查看话题
ros2 topic list | grep imu

# 查看数据
ros2 topic echo /chassis_imu
```

6.2 验证项

测试项	验证方法
数据范围	角度在 $\pm\pi$ 范围内
更新频率	检查消息频率 $\geq 50\text{Hz}$
坐标系	静止时az约为 9.8 m/s^2
CRC校验	发送错误CRC验证丢弃

7. 附录

7.1 字节序

所有多字节值使用 Little-Endian 字节序。

7.2 浮点格式

所有浮点数使用 IEEE 754 单精度 (float32, 4 bytes)。

8. 修订历史

版本	日期	变更内容
1.0.0	2026-01-31	初始版本